

FIELD GUIDE

Workstreams

Parallel, disposable worktrees with a durable ledger

Spin up isolated worktrees for every unit of work, throw them away when they merge, and keep one ledger that always knows where each branch stands.

Version 1 · July 2026

CONTENTS

01 The idea

02 Quickstart

03 The six commands

04 How it works

05 Runbooks

06 Cheat sheet

07 Gotchas & FAQ

The idea

A workstream is one feature split into many small units of work, each living in its own throwaway git worktree.

When a feature is too big for a single branch, you break it into **units** — a slice, a follow-up, a stacked layer. Each unit gets an isolated git worktree and branch of its own, so units never step on each other and can be built, reviewed, and merged independently. The worktree is a pure code checkout: a scratch space you can create, abandon, and recreate at will.

The catch is that a checkout can vanish — dropped, garbage-collected, or replaced — so nothing you care about keeping can live inside it. Every durable datum — a unit's progress and follow-ups, its history of decisions and restacks, and its identity map from unit to repo and branch — lives in a single **global store** under `~/.claude/workstreams/`, one directory per workstream, shared across repos. Volatile facts that git and GitHub already own (current branch, base, PR number, status) are never copied in; they are derived live.

```
~/.claude/workstreams/<ws-id>/
workstream.md      # metadata only
units.md           # append-only ledger (unit ↔ repo/branch identity map)
units/<unit-id>/
  progress.md      # MUTABLE current state: Tasks + Follow-ups checklists
  log.md           # APPEND-ONLY: created, dropped, restack, decision, note
```

Golden rule. The worktree holds only code. Drop and recreate it freely — all progress, history, and identity survive in the store.

Quickstart

Five minutes from nothing to a fresh agent building your first unit in its own worktree — then a board to watch it and a one-command way back in.

The `ws-*` suite tracks multi-unit work in a durable, cross-repo store at `~/ .claude/workstreams/`. A **workstream** is the container; each **unit** is one shippable slice of work with its own worktree and branch. Worktrees are disposable — all durable state lives in the store, so you can drop and recreate a checkout without losing progress.

Golden rule. Worktrees hold code only. Never write store files (`progress.md` , `log.md`) into a worktree — the store, not the checkout, is the source of truth.

- 1 **Create a workstream** with `ws-init` . Give it a name; it computes a dated `ws-id` , scaffolds the store, and reports the id you'll pass to every later command. It creates no units or worktrees — just the container.
- 2 **Spin the first unit** with `ws-start <ws-id> "<what>"` . It mints a globally-unique `unit-id` (`<ws-id>:<slug>`), cuts a `feat-<slug>` branch off the base, and creates a worktree via `wmx` (a per-unit window is optional).
- 3 **Plan and build the unit** with `ws-resume` — from the current session or the unit's worktree. It reads the charter + design, plans tasks as `T1..` into `progress.md` , executes, and appends decisions to `log.md` .
- 4 **Check status** anytime with `ws-board` . It's read-only: it reads the ledger, derives each unit's status live from git and GitHub, and renders a copy-paste-ready board grouped by state.
- 5 **Pick work back up** with `ws-resume <unit-id>` — it finds the unit's worktree, reconciles any base drift, and drops you back into context.

`ws-init` "`<name>`"

Use when you need a new workstream and none exists yet.

Computes `ws-id = <YYYY-MM-DD>-<slug(name)>` , creates `workstream.md` plus an empty ledger, and reports the id. Give it more than one word so it can set the goal without asking.

```
$ /ws-init task templates
→ created workstream 2026-07-03-task-templates
next: ws-start 2026-07-03-task-templates "<what the first unit does>"
```

ws-start <ws-id> "<what>" [--base <unit-id|branch>]

Use when starting a new unit of work in an existing workstream.

Resolves the workstream, mints `unit-id = <ws-id>:<slug>`, and creates the worktree:

```
wmx worktree create feat-<slug> --base <base>
```

It appends the ledger line and seeds `progress.md` and `log.md`; run `ws-resume` next to plan and build. Pass `--base <unit-id>` to stack on another unit's branch (its repo is inherited); `--repo <org/repo>` overrides the cwd repo. Omit `<ws-id>` when exactly one workstream exists.

```
$ /ws-start 2026-07-03-task-templates "template list page"
```

`ws-resume` takes over from here — no bootstrap prompt is printed. It reconstructs the unit's intent from its `charter.md` and the workstream's `design:` spec, plans with **writing-plans** (tasks as `T1..` in `progress.md`), then executes with **subagent-driven-development**, appending decisions to `log.md`. Store files never live in the worktree.

Optional window. Run `ws-resume` in the current session, or open a per-unit `wmx` window for parallel work — if that window opens empty, launch `claude` there yourself.

ws-board [ws-id] [unit-id]

Use when you want to see or share where a workstream stands.

Reads the ledger and derives every unit's status live — no stored status, ever. Status is first-match-wins: dropped, then merged, then in-review (PR ready), then building (draft or no PR). Pass a `unit-id` to print that unit's full checklist plus recent log notes.

```
$ /ws-board 2026-07-03-task-templates
*Task templates* - 0/1 units done
```

```
 In progress • template list page - #4160 (draft) · 2/5 tasks
```

ws-resume <unit-id>

Use when resuming an existing unit, or picking work back up from inside its worktree.

Locates the unit's worktree via its ledger branch, reconciles any base drift (GitHub auto-retargets when a base PR merges), and restores you into the unit's live task state.

```
$ /ws-resume template-list-page
```

Tip. Everything after step 1 keys off the `ws-id` and `unit-id`. If a single workstream exists, most commands let you drop the `ws-id` and infer it.

The six commands

Six skills cover the full life of a unit of work — from spinning up a container to tearing a branch down — while every durable fact lives in the global store, never in a worktree.

Golden rule. Worktrees are disposable code checkouts; all durable state lives in the store. Every command reads the shared contract at `~/.claude/skills/ws-shared/SPEC.md` before it acts, and never stores what git or GitHub already owns (branch, base, PR number, status — all derived live).

ws-init <workstream name>

Use when you need a new workstream and none exists yet — before starting any unit with `ws-start`.

Computes the `ws-id` (`<YYYY-MM-DD>--<slug>`) and creates the store container: `workstream.md` metadata, an empty append-only `units.md` ledger, and the `units/` directory. Sets up the container only — it creates no unit and no worktree.

```
$ /ws-init task templates
```

ws-start <ws-id> <what this unit does> [--base <unit-id|branch>]

Use when starting a new unit of work in an existing workstream — its own worktree plus ledger entry. Run `ws-init` first if no workstream exists.

Mints `unit-id = <ws-id>:<slug>`, resolves the repo (`--repo` › a `--base` unit's repo › cwd) and the base, then creates the worktree via `wmx worktree create feat-<slug> --base <base>`. Appends the ledger line and seeds `progress.md` and `log.md`; `ws-resume` plans and builds.

```
$ /ws-start 2026-07-03-task-templates "add is_template flag" --base schema-migration
```

ws-resume [unit-id]

Use when resuming or continuing an existing unit, or picking work back up from inside its worktree.

With no argument, infers the unit from the current worktree's branch. Ensures the worktree exists (reopening or recreating it if gone), reconciles the base against the recorded one and realigns on drift, then loads `progress.md` and `log.md` and continues at the first unchecked task.

```
$ /ws-resume add-is-template-flag
```

ws-board [ws-id] [unit-id]

Use when you want to see or share where a workstream stands — "show the board", "what's done", "workstream status".

Reads the ledger, derives each unit's status and task counts, and renders copy-paste-ready markdown grouped by state (Done, In progress, Not started, Dropped) with open follow-ups listed across units. Given a `unit-id`, prints that unit's full checklist and recent notes.

```
$ /ws-board 2026-07-03-task-templates
```

Tip. `ws-board` is strictly read-only — it derives everything and reconciles nothing. Reach for it freely to snapshot progress without touching git or GitHub.

ws-restack <unit-id> <new-base>

Use when a unit's base PR merged and its branch must move onto a new base, or GitHub auto-retargeted a dependent PR and the local branch needs realigning.

Resolves `<new-base>` (a branch, or another unit-id → that unit's branch) and rebases per the SPEC reconciliation. When we initiate — the PR's `baseRefName` is unchanged remotely — it runs `gh pr edit --base` first; in the auto case it skips that. Stops and reports on conflicts.

```
$ /ws-restack add-is-template-flag master
```

ws-drop <unit-id>

Use when you want to drop, tear down, abandon, or clean up a unit's worktree and branch.

Shows exactly what will be removed and requires explicit confirmation, then removes the worktree and window and deletes the *local* branch. Appends a `dropped` log line but keeps `progress.md` and the ledger line as the reconciliation record.

```
$ /ws-drop add-is-template-flag
```

Gotcha. `ws-drop` never deletes the remote branch or closes the PR unless you ask, and it preserves the store. To restart, run `ws-start` with the same intent — it versions the id with a `-N` suffix and records `restart-of` automatically.

The durable model

Worktrees are disposable code checkouts; every durable fact lives in the store, and everything volatile is derived live from git and GitHub.

The suite splits every fact about a unit into two camps: what git and GitHub already own, and what only the workstream tracks. It never stores the former — branch, base, PR number, and status are read on demand, so the board can never drift out of sync with reality.

Datum	Where it lives	Note
Current branch	git (the worktree)	<code>git rev-parse --abbrev-ref HEAD</code> — never stored
Base / did GitHub retarget	GitHub	<code>gh pr view --json baseRefName</code> — never stored
PR number + draft/ready/merged	GitHub	<code>gh pr view --json number,isDraft,state</code> — never stored
Status	derived	Computed live from the log + PR state (see precedence below)
Unit ↔ repo/branch identity	<code>units.md</code> ledger	Append-only; set once at <code>ws-start</code> , prior lines never edited
Tasks + completion, follow-ups	<code>progress.md</code>	The only home; mutable work-state SoT
Decisions, notes, drop, restack history	<code>log.md</code>	Append-only history

Status is never written down — it is resolved on every read by walking four rules and taking the **first match**:

- 1 A `dropped` line in `log.md` → `dropped`
- 2 PR merged → `merged`
- 3 PR ready → `in-review`
- 4 PR draft, or no PR → `building`

Mutable state vs append-only history

The two per-unit files answer different questions and never overlap. `progress.md` is the mutable work-state source of truth — its Tasks and Follow-ups checklists always reflect the present, and task/follow-up ids (`T1` , `F1`) are monotonic per unit and never reused, even after check-off or removal. `log.md` is

append-only history — `created`, `dropped`, `restack`, `decision`, `note` — a permanent record you add to but never rewrite.

```
~/claude/workstreams/<ws-id>/
workstream.md      # metadata only
units.md           # append-only ledger (unit ↔ repo/branch)
units/<unit-id>/
  progress.md      # MUTABLE work-state SoT — Tasks + Follow-ups
  log.md           # APPEND-ONLY history
```

The invariant. The log never stores current state; progress never stores history; nothing volatile — branch, base, PR, status — is stored at all. Derive it live.

IDs & base conventions

Identity is deterministic, so the same intent always resolves to the same place:

- 1 **ws-id** = `<YYYY-MM-DD>-<slug(name)>` (from `date -u +%Y-%m-%d`) — this is the store directory name.
- 2 **unit-id** = `<ws-id>:<slug(what)>`, globally unique by construction. A bare `<slug>` works as shorthand — resolved across all workstreams (one match wins; several prompt for the `<ws-id>:` prefix). Re-running the same intent takes a `-N` suffix and records `restart-of` (restart only).
- 3 **branch** = `feat-<slug>` unless the caller supplies one; disambiguated with `-N` on a git-branch collision in the target repo (`:` is illegal in refnames, so the branch is not the canonical id).
- 4 **base** = the repo's default branch, resolved via `gh repo view --json defaultBranchRef -q .defaultBranchRef.name` — unless a base is supplied. A supplied base may name another unit-id, resolving to that unit's branch (stacking).
- 5 **repo** (`ws-start`) resolves by precedence: explicit `--repo` › a `--base` unit's repo › the cwd repo. An explicit `--repo` that contradicts a `--base` unit's repo is an error.

Recorded base, not live base. A unit's recorded base is the base on its last `created` or `restack` line in `log.md` — never the live PR `baseRefName`, which GitHub may have moved when a base PR merged. Restack reconciles the two.

Runbooks

Five recipes for the everyday shape of workstream work — running units side by side, picking one up cold, restarting a dead-end, restacking after a merge, and sharing status.

Golden rule. Worktrees are disposable code checkouts; all durable state lives in the store at `~/.claude/workstreams/<ws-id>/`. Drop and recreate worktrees freely — progress survives because it was never stored in them.

1 · Run two units in parallel

Each unit gets its own worktree off a shared store, so two units run at once without stepping on each other — open a per-unit window if you want them side by side. Start them from the same `ws-id`; the ledger and per-unit `progress.md` keep them separate.

```
ws-start <ws-id> "<what>" [--base <unit-id|branch>] [--repo <org/repo>]
```

Use when starting a new unit in an existing workstream — its own worktree and ledger entry.

Run it once per unit. Each call creates `feat-<slug>` via `wmx worktree create`, appends a ledger line, and seeds that unit's `progress.md` + `log.md`. Units share only the store dir — code stays isolated per worktree.

```
$ /ws-start 2026-07-03-task-templates "template CRUD api"
$ /ws-start 2026-07-03-task-templates "template picker ui"
```

Tip. To stack the second unit on the first instead of running truly parallel, pass `--base <unit-id>`; it resolves to that unit's branch and records `stacked-on` on the ledger line.

2 · Resume a unit cold — new session or next day

Nothing is carried in your head or the shell: `ws-resume` reads the store and reconstructs the unit. Run it from inside the worktree (id inferred from the branch) or name the unit explicitly.

ws-resume [unit-id]

Use when picking work back up — a fresh session, the next day, or after the worktree was dropped.

- 1 Resolves the unit via the bare-slug resolver → `ws-id`, repo, branch (or infers it from the current branch).
- 2 Self-locates the worktree: reuses it in place, `cd` s in if you're elsewhere, reopens via `wmx` if the branch survives, or fresh-starts off the default branch using stored progress as the baseline if the branch is gone.
- 3 Reconciles the base if the live PR base drifted from the recorded base (see recipe 4).
- 4 Loads `progress.md` + recent `log.md`, runs `git log -5` and verification, then continues at the first unchecked task.

```
$ /ws-resume template-crud-api
```

3 · Drop and restart from scratch

Tear the worktree down, then re-run `ws-start` with the same intent — the id versions to `-2` and the old unit is kept as history.

ws-drop <unit-id>

Use when abandoning a unit's worktree and local branch — a dead-end approach you want to redo cleanly.

Shows exactly what it will remove and requires confirmation, then removes the worktree/window and the **local** branch. It appends a `dropped` log line but **keeps** `progress.md` and the ledger line as the reconciliation record. The remote branch and PR stay unless you ask.

```
$ /ws-drop template-crud-api
```

ws-start <ws-id> "<same what>"

Use when restarting the dropped unit fresh.

Re-running with an intent whose slug already exists takes the next `-N` suffix and records `restart-of=<original-unit-id>` on the new ledger line — history stays intact.

```
$ /ws-start 2026-07-03-task-templates "template CRUD api"
# → unit template-crud-api-2 (restart-of=template-crud-api)
```

4 · Restack after a base PR merges

When a unit is stacked on another and that base merges, the dependent branch must move onto its new base. A unit's **recorded base** is the base on its last `created` / `restack` log line — never the live PR base, which GitHub may have moved.

ws-restack <unit-id> <new-base>

Use when a unit's base PR merged, or GitHub auto-retargeted a dependent PR and the local branch needs realigning.

Rebases onto the recorded base via the pinned command, then appends a `restack` line. `<new-base>` may be a branch or another unit-id (resolved to its branch).

```
OLD=$(git merge-base HEAD origin/<recorded-base>)
git fetch
git rebase --onto origin/<new-base> $OLD
```

```
$ /ws-restack template-picker-ui template-crud-api
```

Auto vs we-initiate gate. Compare `gh pr view --json baseRefName` to the recorded base. If it is **unchanged** remotely, we initiate — run `gh pr edit <pr> --base <new-base>` first, then rebase. If it has **already changed** (GitHub auto-retargeted when the base PR merged), skip the `gh pr edit` and just rebase. On conflicts, stop and report.

Tip. Only `ws-restack` (explicit) and `ws-resume` (on detecting drift) reconcile. `ws-board` is read-only and never rebases.

5 · Share status

Print a copy-paste-ready markdown board grouped by state — done, in progress, not started, dropped — with per-unit task counts and open follow-ups.

ws-board [ws-id] [unit-id]

Use when you want to see or share where a workstream stands — "show the board", "what's done".

Derives every status live (nothing stored) and renders markdown you can paste into Slack. With a `unit-id`, it prints that unit's full `progress.md` checklist plus recent `log.md` notes.

Task templates – 1/3 units done

- ✅ Done
 - template CRUD api – #4160
- 🔄 In progress
 - template picker ui – #4162 (draft) · 2/5 tasks
- ⌚ Not started
 - bulk import (deps: template-crud-api)

\$ /ws-board 2026-07-03-task-templates

Cheat sheet

Six skills, one durable store: worktrees hold code, the store holds truth, and status is always derived live from git and GitHub.

Command	Purpose
<code>ws-init</code>	Create a new workstream container (metadata + empty ledger). Run before any unit exists.
<code>ws-start</code>	Start a unit: its own worktree, branch, ledger line, and <code>progress.md</code> / <code>log.md</code> .
<code>ws-resume</code>	Pick a unit back up: reattach the worktree, reconcile base drift, continue at the first unchecked task.
<code>ws-drop</code>	Tear down a unit's worktree and local branch; keep the reconciliation record.
<code>ws-restack</code>	Move a unit's branch onto a new base after the base PR merged or GitHub auto-retargeted.
<code>ws-board</code>	Show or share where a workstream stands. Read-only — derives everything, writes nothing.

```
~/.claude/workstreams/<ws-id>/
├─ workstream.md      # metadata only (id, name, goal, design)
├─ units.md           # append-only ledger: unit ↔ repo/branch identity
└─ units/
   └─ <unit-id>/
      ├─ progress.md  # MUTABLE work-state: Tasks + Follow-ups checklists
      └─ log.md        # APPEND-ONLY: created · dropped · restack · decision · note
```

`ws-id` = `<YYYY-MM-DD>-<slug(name)>` · `unit-id` = `<ws-id>:<slug(what)>` (bare `<slug>` shorthand; `-N` = restart) · `branch` = `feat-<slug>` . The store is global, not per repo.

Worktree = code only. Never write store files into a worktree. Find a unit's worktree via its ledger branch; drop and recreate worktrees freely — progress survives in the store.

progress.md = work-state source of truth (mutable). Tasks and follow-ups live here and nowhere else. Ids (`T1` , `F1`) are monotonic per unit and never reused, even after check-off or removal. No history is kept here.

log.md = append-only. One line per event — `created` , `dropped` , `restack` , `decision` , `note` . Never edit or delete prior lines; the log never stores current state.

Nothing volatile is stored — derive it live. Current branch comes from git, base and retargeting from `gh pr view --json baseRefName`, PR number and draft/ready/merged from GitHub, and status from a first-match cascade: dropped → merged → in-review → building.

Gotchas & FAQ

Worktrees are disposable code checkouts; every durable fact lives in the global store or is derived live from git and GitHub — so most "I lost it" moments are recoverable.

Golden rule. Never store what git or GitHub owns. Branch, base, PR number, and status are always derived live; only tasks, follow-ups, and history are written to the store. If a worktree feels lost, the store still knows the unit.

Question	Answer
Q: I deleted the worktree — is my work lost?	A: No. Worktrees are disposable; all durable state lives in <code>~/ .claude/workstreams/<ws-id>/</code> . Run <code>/ws-resume <unit-id></code> ; if the branch still exists it reopens the worktree; if the branch is also gone it starts fresh off the repo default, using the store's <code>progress.md</code> as the restart baseline.
Q: Which unit am I in?	A: Run <code>/ws-resume</code> with no argument. It infers the unit from the current worktree's branch by scanning every <code>units.md</code> ledger in the store, then resolves the <code>ws-id</code> , repo, and branch for you.
Q: My base PR merged — how do I move onto the new base?	A: Run <code>/ws-restack <unit-id> <new-base></code> . It rebases <code>--onto</code> the new base from the recorded merge-base and appends a <code>restack</code> line to <code>log.md</code> . When you initiate and the PR base is unchanged remotely, it also runs <code>gh pr edit --base</code> first.
Q: GitHub auto-retargeted my dependent PR — do I still restack?	A: The realign still happens, but the <code>gh pr edit</code> step is skipped since GitHub already moved the base. <code>/ws-resume</code> also auto-reconciles: if the live <code>baseRefName</code> differs from the recorded base, it realigns and records a <code>restack</code> line before continuing.
Q: I opened a per-unit window and it didn't auto-start the agent.	A: Windows are optional — run <code>ws-resume</code> in your current session, or launch <code>claude</code> in the window yourself. wmx only autostarts the agent when configured to.
Q: Can a workstream span multiple repos?	A: Yes. The store is global, not per-repo. Each unit records its own <code>repo=<org/repo></code> (and branch) on its ledger line in <code>units.md</code> , so units of one workstream can live in different repositories.

Optional window. A per-unit window is optional — run `ws-resume` in your current session instead. If you do open one and it doesn't start the agent, run `claude` manually (wmx only autostarts when configured to).

Tip. Read-only by design: `ws-board` never reconciles a base. Only `ws-restack` (explicit) and `ws-resume` (on detecting drift) move a branch onto a new base.